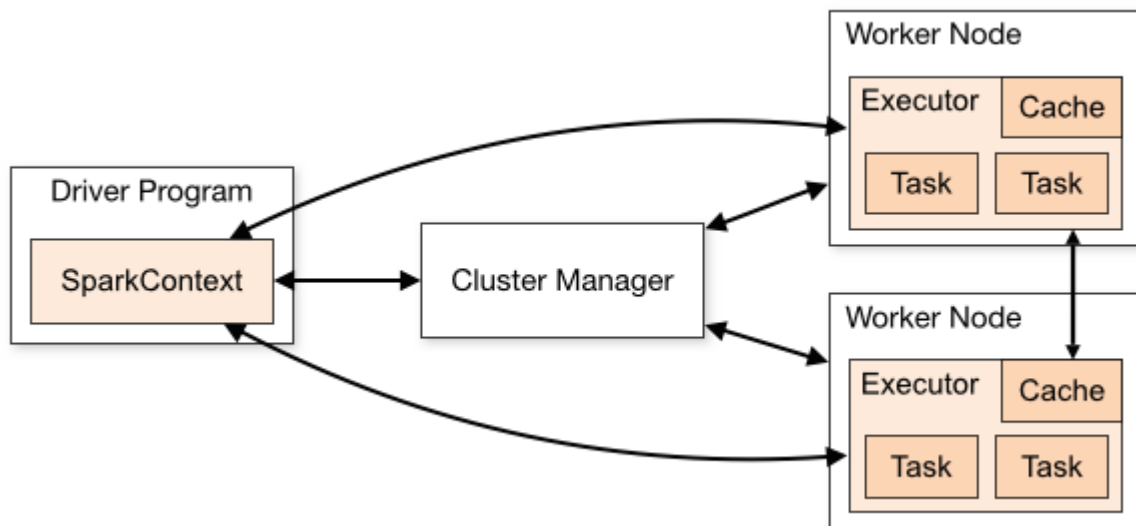




Spark uses a master/slave architecture. As you can see in the figure, it has one central coordinator (Driver) that communicates with many distributed workers (executors). The driver and each of the executors run in their own Java processes.

DRIVER

The driver is the process where the main method runs. First it converts the user program into tasks and after that it schedules the tasks on the executors.

EXECUTORS

Executors are worker nodes' processes in charge of running individual tasks in a given Spark job. They are launched at the beginning of a Spark application and typically run for the entire lifetime of an application. Once they have run the task they send the results to the driver. They also provide in-memory storage for RDDs that are cached by user programs through Block Manager.

APPLICATION EXECUTION FLOW

With this in mind, when you submit an application to the cluster with `spark-submit` this is what happens internally:

1. A standalone application starts and instantiates a `SparkContext` instance (and it is only then when you can call the application a driver).
2. The driver program ask for resources to the cluster manager to launch executors.
3. The cluster manager launches executors.

4. The driver process runs through the user application. Depending on the actions and transformations over RDDs task are sent to executors.
 5. Executors run the tasks and save the results.
 6. If any worker crashes, its tasks will be sent to different executors to be processed again.
 7. Spark automatically deals with failed or slow machines by re-executing failed or slow tasks. For example, if the node running a partition of a map() operation crashes, Spark will rerun it on another node; and even if the node does not crash but is simply much slower than other nodes, Spark can preemptively launch a “speculative” copy of the task on another node, and take its result if that finishes.
 7. With SparkContext.stop() from the driver or if the main method exits/crashes all the executors will be terminated and the cluster resources will be released by the cluster manager.
-
1. When executors are started they register themselves with the driver and from so on they communicate directly. The workers are in charge of communicating the cluster manager the availability of their resources.
 2. In a YARN cluster you can do that with --num-executors. In a standalone cluster you will get one executor per worker unless you play with spark.executor.cores and a worker has enough cores to hold more than one executor.
 3. You can assign the number of cores per executor with --executor-cores
 4. --total-executor-cores is the max number of executor cores per application

Spark is lazy programming and RDD's are the primary abstraction which are resilient and fault tolerant in nature.

RDD performs two basic operations :

1> Transformations

2> Actions

Both together composed an spark application which are divided into serializable tasks by driver and sent to the worker node with respect to the number of partitions.

When the series of Transformations are done on RDD's it stores every transformed RDD in **new RDD** at all the worker nodes where partitions are. All the Transformations are updated in Direct ACyclic Graph(DAG) which exhibits the RDD lineage.

When actions are called on these RDD's which are partitioned among the different nodes parallel processing occurs at every worker node by executor and **returns results from each node to the driver once the tasks get completed.**